

MOCHI_API_KEY 專案調查報告

Codex

2026-03-07

摘要

本次調查的核心問題不是「Mochi 官方是否已廢棄 API key」，而是「本專案是否還把 MOCHI_API_KEY 當作系統層環境變數來使用」。

結論如下：

1. 若以 **Mochi** 官方產品來看，Mochi API key 並未被官方廢棄。
2. 若以本專案的主要執行路徑來看，系統層 `.env` 中的 MOCHI_API_KEY 已基本退場。
3. 本專案目前真正生效的主路徑，是 **per-user** 設定中的 `mochi_api_key`。
4. **repo** 內仍殘留多條 **legacy** 路徑與文件，導致 MOCHI_API_KEY 看起來像還在被系統層使用。
5. 目前資料結構存在不一致：**API runtime** 讀 `config.mochi_api_key`，但部分本地資料與 **CLI** 還在讀頂層 `mochi_api_key`。

調查範圍

- iOS app 設定畫面如何讀寫 Mochi key
- KG backend runtime 如何取得 Mochi key
- CLI / script / `.env` 是否仍直接讀 MOCHI_API_KEY
- **repo** 內文件與現況是否一致
- 是否存在安全風險

外部事實：Mochi 官方並未廢棄 API key

官方文件與公開頁面顯示，Mochi 仍維持 API key 機制：

- 官方 API 文件仍明確寫出：Mochi 使用 API keys 進行驗證，並以 HTTP Basic Auth 傳入。
- 官方 2025-10-08 部落格文章仍提到，可從 Account Settings 產生 API key。
- 直接請求 `https://app.mochi.cards/api/decks` 會回 403，顯示 API 端點仍存活，並非移除或失效。

參考來源：

- Mochi API Reference: <https://mochi.cards/docs/api/>
- Mochi blog, *New Browser Add-ons and CSV exports* (2025-10-08): <https://mochi.cards/blog/new-browser-add-ons/>

因此，若說 MOCHI_API_KEY 被「官方廢棄」，目前沒有證據支持。

專案內部事實：主要 runtime 已不再依賴系統層 MOCHI_API_KEY

1. iOS app 透過 user config 讀寫 Mochi key

iOS 端在登入後會先讀取使用者設定，將 `config.mochi_api_key` 取回，並在輸入欄位變更後透過 API 寫回：

- `SettingsView` 進入畫面時呼叫 `kgService.fetchUserConfig()`
- 將回傳的 `config.mochi_api_key` 指派到 `mochiApiKey`
- 使用者修改後呼叫 `kgService.updateUserConfig(mochiKey: ...)`

關鍵檔案：

- `booksbrowser_ios/BooksBrowser/Views/Settings/SettingsView.swift`
- `booksbrowser_ios/BooksBrowser/Services/KGService.swift`

2. backend runtime API 也是走 per-user config

後端 API 在 `GET /api/user/config` 與 `PUT /api/user/config` 中，明確讀寫的是使用者資料中的 `config.mochi_api_key`。

此外，在 pipeline 的 Mochi sync 步驟中，runtime 也讀的是：

```
user["config"].get("mochi_api_key")
```

而不是 `os.getenv("MOCHI_API_KEY")`。

關鍵檔案：

- `knowledge_graph_api/src/kg/api.py`

這代表從 app 設定頁進來的正常使用流程，主要依賴的是 **per-user config**，而不是系統層 `.env`。

仍存在的 legacy 路徑

雖然主要 runtime 已轉向 per-user config，但 repo 內仍存在多條遺留用法：

1. .env 與 .env.example 仍保留 MOCHI_API_KEY

`knowledge_graph_api/.env.example` 仍列出：

- `MOCHI_API_KEY=...`

這會讓維護者直覺以為系統層仍需要它。

2. 維護腳本仍直接讀 os.getenv("MOCHI_API_KEY")

以下腳本仍直接從環境變數讀取：

- `knowledge_graph_api/scripts/reset_system.py`
- `knowledge_graph_api/scripts/migrate_to_apple.py`

這表示「系統層 `MOCHI_API_KEY` 已不再是主 runtime 路徑」，但尚未從所有維護腳本中完全清除。

3. iOS 文案與說明仍把它描述成一般設定項

設定頁與說明 sheet 仍鼓勵使用者手動填入 Mochi API Key，這件事本身沒有錯，但它延續了這個整合仍是活躍功能的產品語意。

重要發現：資料結構存在不一致

這次調查中最重要的工程發現，是專案內對 `mochi_api_key` 的資料形狀並不一致。

API runtime 期望的格式

在 `knowledge_graph_api/src/kg/api.py` 中，`get_current_user()` 回傳：

- `record`
- `config: record.get("config", {})`

後續 runtime 讀 Mochi key 時，是從：

- `user["config"].get("mochi_api_key")`

取得。

這代表 API runtime 期望的資料形狀是：

```
{
  "some_user": {
    "config": {
      "mochi_api_key": "...
    }
  }
}
```

CLI 與部分本地資料仍在讀頂層欄位

但 `knowledge_graph_api/src/kg/cli.py` 仍在讀：

- `users.get(user_id, {}).get("mochi_api_key")`

也就是頂層欄位，而不是 `config.mochi_api_key`。

同時，本地 `knowledge_graph_api/data/users.json` 目前也出現了頂層 `mochi_api_key` 形式。

因此，現在的實際狀態不是單純「從系統 env 改成 per-user DB」而已，而是：

- **API runtime：讀 nested config**
- **CLI / 舊資料：仍可能讀 top-level key**

這是一個結構不一致問題，可能導致：

- app / API 看不到某些既有 Mochi key
- CLI 和 API 對同一位 user 的讀值結果不同
- 維護者誤判哪一條路徑才是權威來源

安全風險

本次調查中還發現一個更高優先級的問題：

repo 內存在已儲存的使用者 Mochi key

knowledge_graph_api/data/users.json 內存在 mochi_api_key 欄位。報告撰寫時未揭露其值，但從安全角度看，這代表敏感憑證目前已進入 **repo workspace** 的一般檔案。

即使這是本地開發資料，也有以下風險：

- 被誤同步、誤備份、誤分享
- 被其他工具或 agent 掃描到
- 導致真正的使用者憑證外洩

建議將該 key 視為已暴露憑證處理，至少進行：

1. 旋轉 / 重發
2. 從 repo 相關資料檔移除
3. 改用明確的 secrets storage 或非版本化資料目錄

結論

最準確的說法不是：

MOCHI_API_KEY 被官方廢棄

而是：

在本專案中，系統層 MOCHI_API_KEY 已不再是主要 runtime 來源；主要使用路徑已轉為 per-user mochi_api_key。但 repo 內仍保留 legacy env、腳本、文件與 CLI 路徑，且資料結構尚未完全統一。

建議行動

A. 決定唯一權威來源

建議明確定義：

- 權威來源 = `users[user_id].config.mochi_api_key`

並停止使用其他資料形狀。

B. 修正 CLI 與 runtime 對齊

將 knowledge_graph_api/src/kg/cli.py 改為讀：

- `users.get(user_id, {}).get("config", {}).get("mochi_api_key")`

避免和 API runtime 分叉。

C. 將系統層 MOCHI_API_KEY 降級為 legacy-only

若只剩維護腳本需要，建議：

- 在文件中標註為 legacy / admin-only
- 不再放在主 .env.example 的核心變數列表

D. 清理文件與 UI 文案

如果 Mochi 整合仍保留，就讓文件明確寫成：

- 使用者層設定，不是系統層設定

如果 Mochi 整合準備退場，就應同步收掉：

- iOS 設定頁欄位
- 說明 sheet
- README 與 .env.example 中的描述

E. 立即處理已暴露的憑證

建議優先執行：

1. 旋轉 Mochi key
2. 清理 knowledge_graph_api/data/users.json
3. 檢查是否有其他備份、副本、匯出檔保留舊 key

附錄：本次核對的主要檔案

- knowledge_graph_api/src/kg/api.py
- knowledge_graph_api/src/kg/cli.py
- knowledge_graph_api/scripts/reset_system.py
- knowledge_graph_api/scripts/migrate_to_apple.py
- knowledge_graph_api/.env.example
- knowledge_graph_api/data/users.json
- booksbrowser_ios/BooksBrowser/Views/Settings/SettingsView.swift
- booksbrowser_ios/BooksBrowser/Services/KGService.swift